

Requirements Gap Analysis Agent

End-to-End Pipeline Interview Guide

One-line explanation

This system reads business and engineering transcripts, extracts requirements and solutions, retrieves the most relevant implementation evidence with open-source embeddings, then uses a local open-source LLM such as Qwen3-32B to analyze and verify gaps.

What you can say in an interview

- I designed the pipeline for long real-world transcripts, not only short demo text.
- The system uses chunking to avoid LLM context overflow.
- It uses BAAI/bge-m3 embeddings for semantic retrieval.
- It uses Qwen3-32B through Ollama for extraction, gap analysis, and critic verification when qwen_local.yaml is selected.
- The output is a structured GapReport that can be written to Markdown, JSON, or JSONL.

1. What Problem This Solves

Business teams describe expectations in meetings, while engineering teams describe what was implemented, deferred, or scoped out. The agent turns both sides into structured evidence and produces a verified requirements gap report.

2. High-Level Architecture

Input	Step 1	Step 2	Step 3 / Output
Business transcripts	Parser	Chunker	Qwen requirement extraction
Engineering transcripts	Parser	Chunker	Qwen solution extraction
Extracted candidates	Deduplication	BGE-M3 embeddings	Top-k retrieval
Requirement + candidates	Qwen focused gap analysis	Qwen critic	Report writers

Core idea

Do not compare every requirement with every engineering statement. Retrieve the most relevant candidate solutions first, then let Qwen reason over a focused evidence set.

3. Data Objects

Object	Meaning
Transcript	One source file with speaker-tagged lines and original line numbers.
Chunk	A small overlapping transcript window used for safe LLM calls.
Requirement	A business need with statement, quote, line range, constraints, notes, and confidence.
Solution	An engineering implementation or scope statement with tech stack, deferred work, and limits.
RetrievalMatch	A cosine similarity match between one requirement and one solution.
Gap	A verified finding: unaddressed, partial, or covered; only non-covered gaps are reported.

4. Each Pipeline Stage

#	Stage	What happens	Why it matters
1	Load transcripts	Reads .txt, .md, and .jsonl files recursively.	Keeps source path, speaker, and line numbers.
2	Chunk transcripts	Splits long files into overlapping windows.	Avoids context overflow and preserves nearby context.
3	Extract requirements	Qwen turns business chunks into structured Requirement objects.	Makes messy conversation machine-readable.
4	Extract solutions	Qwen turns engineering chunks into structured Solution objects.	Captures built, deferred, and out-of-scope work.
5	Deduplicate	Merges repeated items caused by overlap.	Keeps the report clean and readable.
6	Embed text	BGE-M3 converts requirements and solutions into vectors.	Enables semantic search instead of keyword-only matching.
7	Retrieve top-k	For each requirement, find the top 5 likely solutions.	Sends Qwen only focused evidence.
8	Analyze gaps	Qwen compares one requirement with candidate solutions.	Classifies unaddressed or partial coverage.
9	Critic verify	Qwen reviews candidate gaps for false positives.	Improves trust in the final report.
10	Write report	Outputs Markdown, JSON, or JSONL.	Makes results useful for teams and review.

5. Standard Mode vs Large Qwen Mode

Area	Standard mode	Large Qwen mode
Command	Default when --mode is omitted	--mode large --config configs/qwen_local.yaml
Best for	Small tests and fallback demo	Large transcript datasets
Extraction	Heuristic in default config	Qwen3-32B through Ollama
Embeddings	Hashing fallback	BAAI/bge-m3
Gap reasoning	Deterministic rules	Qwen focused comparison
Failure behavior	Always runnable	Fails clearly if Qwen is unavailable

6. Interview Script

30-second answer

I built a requirements gap analysis agent for long transcripts. It chunks business and engineering conversations, uses Qwen to extract structured requirements and solutions, deduplicates overlap, embeds the items with BGE-M3, retrieves top-k matching solutions for each requirement, and then asks Qwen to perform focused gap analysis and critic verification before writing a report.

If asked why chunking is needed

- LLMs have context limits.
- Long transcripts can exceed those limits.
- Overlap prevents losing context at chunk boundaries.

If asked why embeddings are needed

- They avoid comparing every requirement to every solution.
- They retrieve semantically related evidence.
- They reduce cost, latency, and irrelevant context.

If asked what is open source

- Qwen3-32B is used locally through Ollama.
- BAAI/bge-m3 is used through sentence-transformers.
- The cosine vector index is implemented locally, so FAISS is optional.

7. Strengths, Tradeoffs, and Future Work

Strength	Tradeoff	Future improvement
Scales beyond one context window	More chunks means more model calls	Batching and async processing
Focused evidence is explainable	Retrieval can miss weak matches	Hybrid keyword + vector retrieval
Structured schemas are testable	LLM JSON can need validation	Retry and repair loop
Local open-source stack	Qwen 32B needs strong hardware	Model profiles: 8B, 14B, 32B
Modular design	No graph reasoning yet	GNN or knowledge graph as future work

8. Commands to Run

Action	Terminal command
Install dependencies	<code>pip install -r requirements.txt</code>
Pull Qwen model	<code>ollama pull qwen3:32b</code>
Run large Qwen mode	<code>python main.py --mode large --config configs/qwen_local.yaml --business transcripts/business --engineering transcripts/engineering --output reports/large_qwen_report.md --format markdown</code>
Run tests	<code>python -m pytest -q</code>

Final closing line

The key design decision is retrieval before reasoning: embeddings narrow the evidence, and Qwen reasons over focused pairs instead of the entire transcript corpus.